arXiv:2401.10020v3 [cs.CL] 28 Mar 2025

Self-Rewarding Language Models

Weizhe Yuan[1,2] **Richard Yuanzhe Pang**[1,2] **Kyunghyun Cho**[2]
Xian Li[1] **Sainbayar Sukhbaatar**[1] **Jing Xu**[1] **Jason Weston**[1,2]

[1] Meta [2] NYU

Abstract

We posit that to achieve superhuman agents, future models require superhuman feedback in order to provide an adequate training signal. Current approaches commonly train reward models from human preferences, which may then be bottlenecked by human performance level, and secondly these separate frozen reward models cannot then learn to improve during LLM training. In this work, we study *Self-Rewarding Language Models*, where the language model itself is used via LLM-as-a-Judge prompting to provide its own rewards during training. We show that during Iterative DPO training that not only does instruction following ability improve, but also the ability to provide high-quality rewards to itself. Fine-tuning Llama 2 70B on three iterations of our approach yields a model that outperforms many existing systems on the AlpacaEval 2.0 leaderboard, including Claude 2, Gemini Pro, and GPT-4 0613. While there is much left still to explore, this work opens the door to the possibility of models that can continually improve in both axes.

1 Introduction

Aligning Large Language Models (LLMs) using human preference data can vastly improve the instruction following performance of pretrained models [Ouyang et al., 2022, Bai et al., 2022a]. The standard approach of Reinforcement Learning from Human Feedback (RLHF) learns a reward model from these human preferences. The reward model is then frozen and used to train the LLM using RL, e.g., via PPO [Schulman et al., 2017]. A recent alternative is to avoid training the reward model at all, and directly use human preferences to train the LLM, as in Direct Preference Optimization [DPO; Rafailov et al., 2023]. In both cases, the approach is bottlenecked by the size and quality of the human preference data, and in the case of RLHF the quality of the frozen reward model trained from them as well.

In this work, we instead propose to train a self-improving reward model that, rather than being frozen, is continually updating during LLM alignment, in order to avoid this bottleneck. The key to such an approach is to develop an agent that possesses all the abilities desired during training, rather than separating them out into distinct models such as a reward model and a language model. In the same way that pretraining and multitasking training of instruction following tasks allow task transfer by training on many tasks at once [Collobert and Weston, 2008, Radford et al., 2019, Ouyang et al., 2022], incorporating the reward model into that same system allows task transfer between the reward modeling task and the instruction following tasks.

We thus introduce *Self-Rewarding Language Models*, that both (i) act as instruction following models generating responses for given prompts; and (ii) can generate and evaluate new instruction following examples to add to their own training set. We train these models using an Iterative DPO framework similar to that recently introduced in Xu et al. [2023].

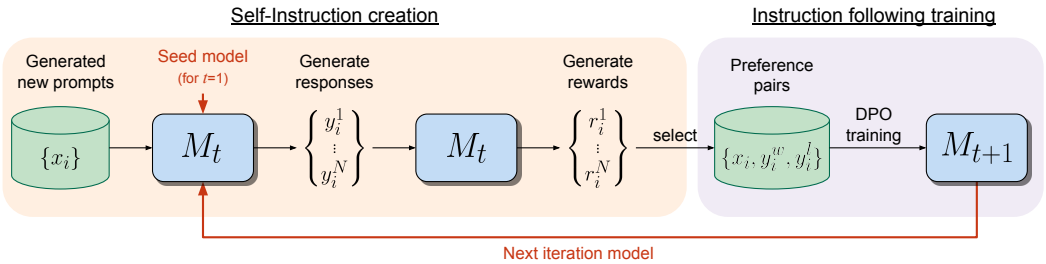

Figure 1: **Self-Rewarding Language Models.** Our self-alignment method consists of two steps: (i) *Self-Instruction creation*: newly created prompts are used to generate candidate responses from model M_t, which also predicts its own rewards via LLM-as-a-Judge prompting. (ii) *Instruction following training*: preference pairs are selected from the generated data, which are used for training via DPO, resulting in model M_{t+1}. This whole procedure can then be iterated resulting in both improved instruction following and reward modeling ability.

Starting from a seed model, in each iteration there is a process of *Self-Instruction creation* whereby candidate responses are generated by the model for newly created prompts, and are then assigned rewards by that same model. The latter is implemented via LLM-as-a-Judge prompting, which can also be seen as an instruction following task. A preference dataset is built from the generated data, and the next iteration of the model is trained via DPO, see Figure 1.

In our experiments, we start with a Llama 2 70B [Touvron et al., 2023] seed model fine-tuned on Open Assistant [Köpf et al., 2023], and then perform the above training scheme. We find that not only does the instruction following performance improve from Self-Rewarding LLM alignment compared to the baseline seed model, but importantly the reward modeling ability, which is no longer fixed, improves as well. This means that the model during iterative training is able, at a given iteration, to provide a higher quality preference dataset to itself than in the previous iteration. While this effect likely saturates in real-world settings, it provides the intriguing possibility of obtaining reward models (and hence LLMs) that are superior to ones that could have been trained from the original human-authored seed data alone.

2 Self-Rewarding Language Models

Our approach first assumes access to a base pretrained language model, and a small amount of human-annotated seed data. We then build a model that aims to possess two skills simultaneously:

1. *Instruction following*: given a prompt that describes a user request, the ability to generate a high quality, helpful (and harmless) response.
2. *Self-Instruction creation*: the ability to generate and evaluate new instruction-following examples to add to its own training set.

These skills are used so that the model can perform self-alignment, i.e., they are the components used to iteratively train itself using AI Feedback (AIF).

Self-instruction creation consists of generating candidate responses and then the model itself judging their quality, i.e., it acts as its own reward model, replacing the need for an external one. This is implemented via the *LLM-as-a-Judge* mechanism [Zheng et al., 2023b], i.e., by formulating the evaluation of responses as an instruction following task. This self-created AIF preference data is used as a training set.

Our overall *self-alignment* procedure is an iterative one, which proceeds by building a series of such models, with the aim that each improves over the last. Importantly, because the model can both improve its generation ability, and act as its own reward model through the same generation mechanism, this means the reward model itself can improve through these iterations, deviating from standard practices where the reward model is fixed [Ouyang et al.,

2022]. We believe this can increase the ceiling of the potential for self-improvement of these learning models going forward, removing a constraining bottleneck.

We describe these steps in more detail below. An overview of the approach is illustrated in Figure 1.

2.1 Initialization

Seed instruction following data We are given a seed set of human-authored (instruction prompt, response) general instruction following examples that we use for training in a supervised fine-tuning (SFT) manner, starting from a pretrained base language model. Subsequently this will be referred to as Instruction Fine-Tuning (IFT) data.

Seed LLM-as-a-Judge instruction following data We also assume we are provided a seed set of (evaluation instruction prompt, evaluation result response) examples which can also be used for training. While this is not strictly necessary, as the model using IFT data will already be capable of training an LLM-as-a-Judge, we show that such training data can give improved performance (see Appendix A.3 for supporting results). In this data, the input prompt asks the model to evaluate the quality of a given response to a particular instruction. The provided evaluation result response consists of chain-of-thought reasoning (a justification), followed by a final score (in our experiments out of 5). The exact prompt format we chose is given in Figure 2, which instructs the LLM to evaluate the response using five additive criteria (relevance, coverage, usefulness, clarity and expertise), covering various aspects of quality. Subsequently this will be referred to as Evaluation Fine-Tuning (EFT) data.

We use both these seed sets together during training.

2.2 Self-Instruction Creation

Using the model we have trained, we can make it self-modify its own training set. Specifically, we generate additional training data for the next iteration of training.

This consists of the following steps:

1. Generate a new prompt: We generate a new prompt x_i using few-shot prompting, sampling prompts from the original seed IFT data, following the approach of Wang et al. [2023] and Honovich et al. [2023].[1]
2. Generate candidate responses: We then generate N diverse candidate responses $\{y_i^1, \ldots, y_i^N\}$ for the given prompt x_i from our model using sampling.
3. Evaluate candidate responses: Finally, we use the LLM-as-a-Judge ability of our same model to evaluate its own candidate responses with scores $r_i^n \in [0, 5]$ (exact prompt given in Figure 2).

2.3 Instruction Following Training

As previously described, training is initially performed with the seed IFT and EFT data (Section 2.1). This is then augmented with additional data via AI (Self-)Feedback.

AI Feedback Training After performing the self-instruction creation procedure, we can augment the seed data with additional examples for training, which we refer to as AI Feedback Training (AIFT) data.

To do this, we construct *preference pairs*, which are training data of the form (instruction prompt x_i, winning response y_i^w, losing response y_i^l). To form the winning and losing pair we take the highest and lowest scoring responses from the N evaluated candidate responses (see

[1] In our main experiments, responses and rewards, items (2) and (3), are generated by the model we have trained, but generating prompts is actually done by a model fixed in advance. However, we show that prompts can also be generated by the newly trained model in each iteration in Appendix A.5.